COMSATS UNIVERSITY ISLAMABAD · LAHORE CAMPUS

CSC241 · LECTURE 15

Every Java Object

Equality, Identity & Copying

From inherited Object methods to safe independent copies

Dr Muhammad Shahid Bhatti

Department of Computer Science | Fall 2026

A deceptively simple question

Two objects can have identical data but different identities.

```
Student a = new Student("Ali", 20);
Student b = new Student("Ali", 20);

System.out.println(a == b);
System.out.println(a.equals(b));
```

Question 1

Do a and b refer to one object or two?

Question 2

Will both expressions produce the same output?

Do not answer yet

Learning goals & 90-minute route

01

Object as a superclass

Core methods: toString, equals, hashCode, getClass

02

Identity and equality

Reference identity, value equality, equality rules

03

Making copies safely

Aliasing, shallow copying, deep copying, copy constructors

Think → predict → execute → explain

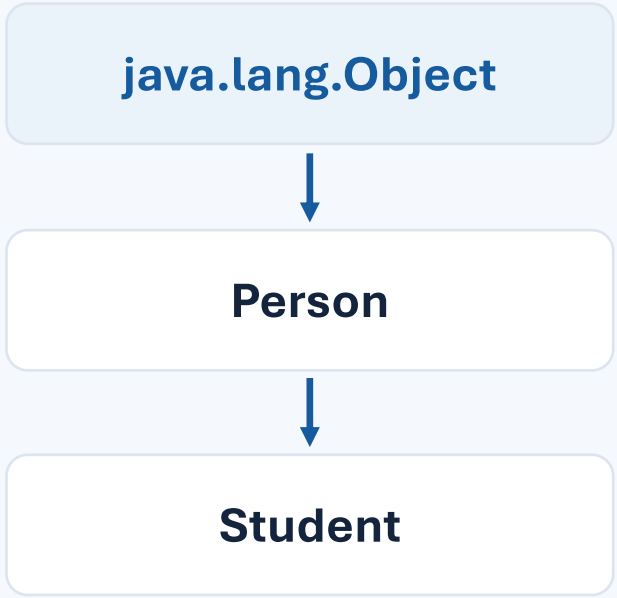
Every Java class ultimately extends Object

Inheritance is not just the hierarchy we write: Java provides one above it.

Java inserts it

class Person { ... }

acts as though it extends Object.



The implication

Student inherits methods defined in Object unless overridden.

Which Object methods matter here?

Not all inherited methods are equally important in an introductory OOP course.

toString()

How should this object look as text?

equals(Object)

What does “same value” mean?

hashCode()

A value used in hash-based collections

getClass()

What is the runtime class?

clone()

Low-level copying mechanism

wait / notify

Thread coordination — later course

toString(): what printing actually asks for

```
class Course {
    String code = "CSC241";
}

Course c = new Course();
System.out.println(c);
```

Default output

Course@<hex hash>
(example; exact suffix varies)

What happened?

println(Object) obtains a string representation using toString().

Override toString() to make objects readable

```
class Course {
    private String code;
    private int credits;

    @Override
    public String toString() {
        return "Course{" + code
            + ", credits=" + credits + "}";
    }
}
```

Result

Course{CSC241, credits=3}

Design goal

Human-readable and useful for debugging.

Predict the output: implicit toString()

```
Course c = new Course("CSC241", 3);
System.out.println(c);
System.out.println("Selected: " + c);
System.out.println(c.toString());
```

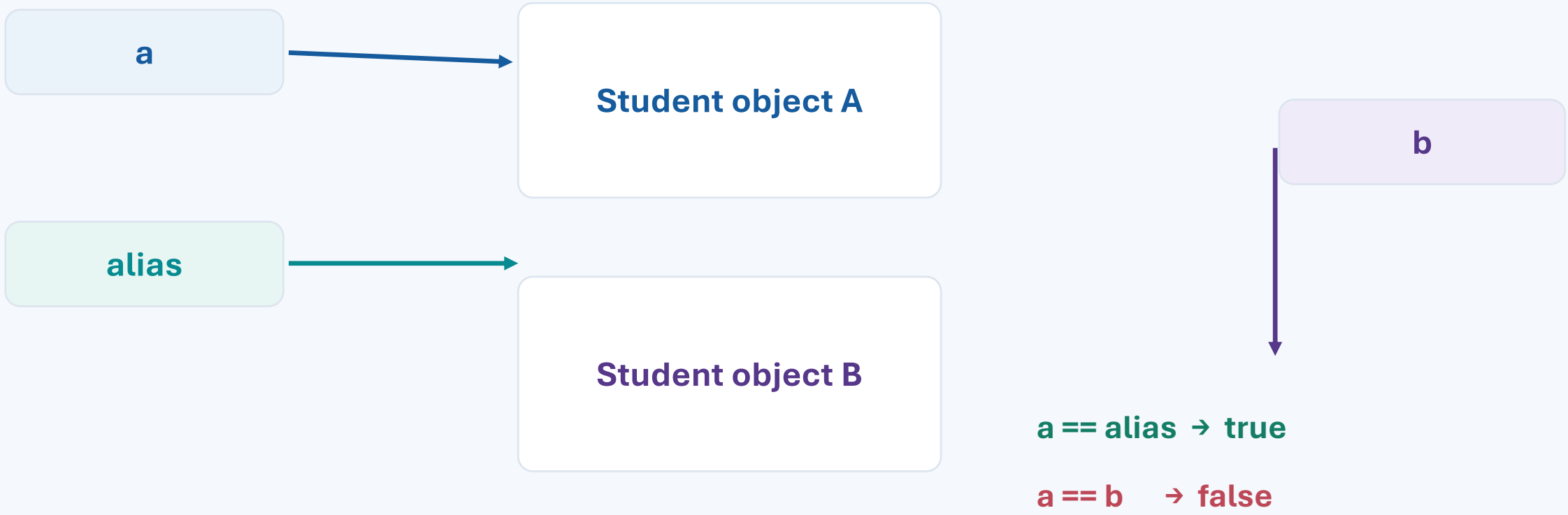
All three expressions use the same textual representation.

SECTION 02

Identity is not equality

A reference answers “which object?”; an equality rule answers “same meaningful value?”

== compares references for object types



Object.equals() initially behaves like ==

```
Student a = new Student("Ali", 20);
Student b = new Student("Ali", 20);
Student alias = a;

System.out.println(a == b);
System.out.println(a.equals(b));
System.out.println(a == alias);
```

Output

false
false
true

Unless Student overrides equals(Object).

Designing value-based equality

1. **Same reference?** Return true immediately.
2. **Compatible object?** Reject null or unrelated types.
3. **Compare meaningful fields** For StudentId, registration number is the identity.

Implement equals(Object) correctly

Here, the final StudentId class represents one logical registration identifier.

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof StudentId)) return false;

    StudentId other = (StudentId) obj;
    return Objects.equals(this.value, other.value);
}
```

Import: java.util.Objects | Use the signature equals(Object), not equals(StudentId).

The equals() contract

Reflexive	x.equals(x) must be true
Symmetric	x.equals(y) == y.equals(x)
Transitive	If x = y and y = z, then x = z
Consistent	Unchanged state → repeatable result
Null-safe	x.equals(null) is false

String is the classic equality trap

```
String x = new String("Java");
String y = new String("Java");

System.out.println(x == y);
System.out.println(x.equals(y));
```

Output

false
true

For object data, use equals when the logical value matters.

equals() and hashCode() are a pair

Required

If a.equals(b) is true, then a.hashCode() == b.hashCode().

Not required

Matching hash codes do NOT prove two objects are equal.

Equal values → **Equal hash codes** (one-way guarantee)

Finish the equality implementation

```
@Override
public int hashCode() {
    return Objects.hash(value);
}

@Override
public String toString() {
    return "StudentId[" + value + "]";
}
```

Equals criterion

Registration number only

Hash criterion

The exact same field

getClass(): ask an object its actual class

```
Course c = new Course("CSC241", 3);

System.out.println(c.getClass().getName());
System.out.println(c.getClass().getSimpleName());
System.out.println(c instanceof Object);
```

Typical output

Course | Course | true (unnamed package)

Subclasses make equality more delicate

Potential pitfall

A Person might compare only a name, while Student compares name and registration number. Can their equality rules disagree?

Safe starting policy

For immutable identifier/value classes, prefer a final class and compare one stable set of fields.

When inheritance enters equals(), symmetry and transitivity need deliberate design.

SECTION 03

One reference or an independent copy?

Correct code depends on knowing whether mutable nested objects are shared.

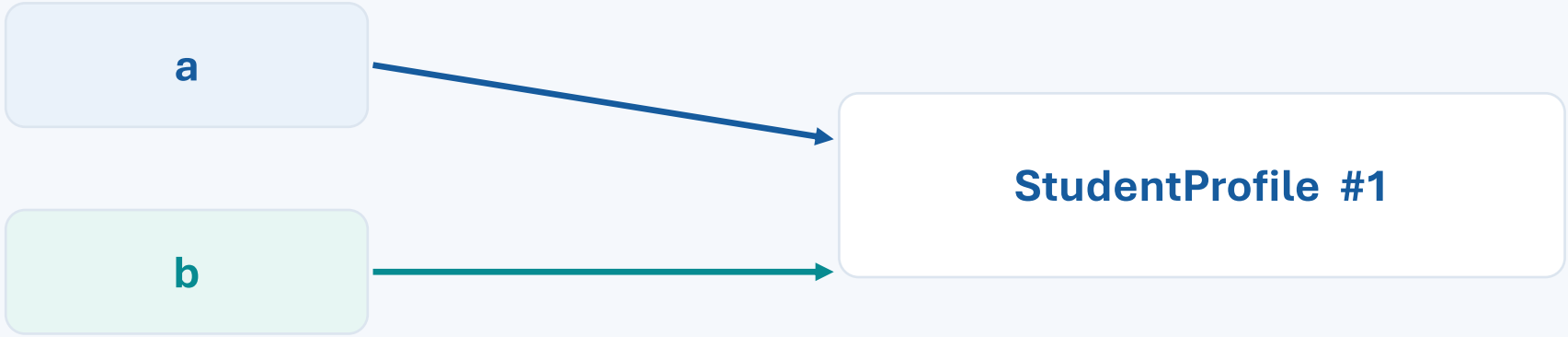
Three different operations

The names are similar; the object graphs are not.

Alias	`b = a`	Same outer object; all state shared
Shallow copy	`b = new Person(a.name, a.address)`	New outer object; inner Address shared
Deep copy	`b = new Person(a)`	New outer object and independent Address

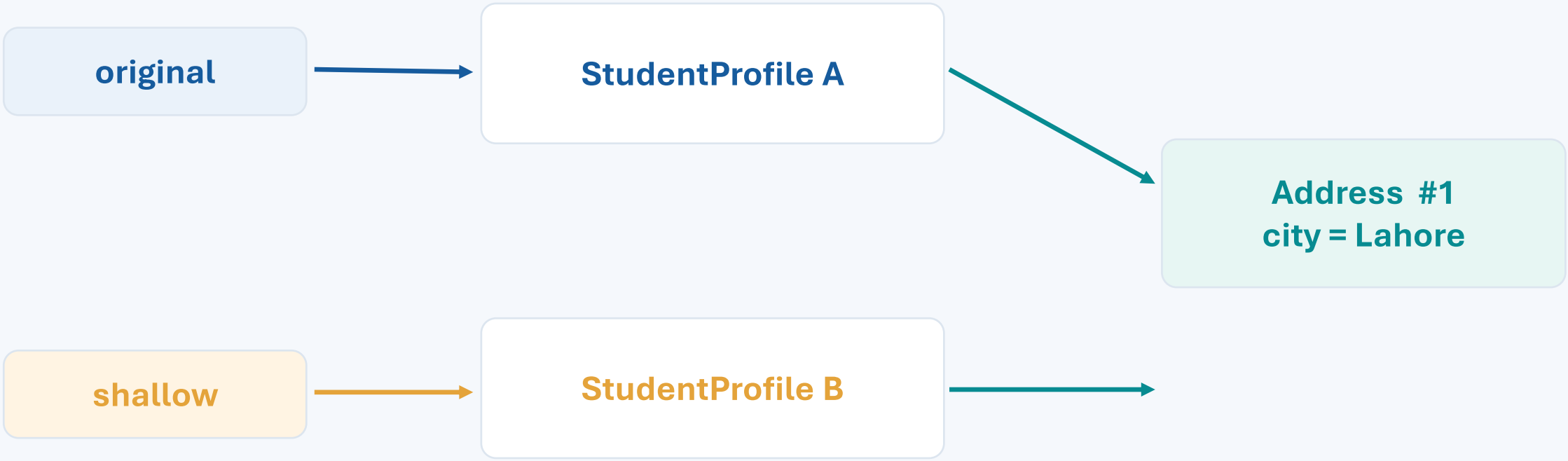
Assignment creates an alias, not a copy

```
StudentProfile a = new StudentProfile(...);
StudentProfile b = a;
System.out.println(a == b);   // true
```



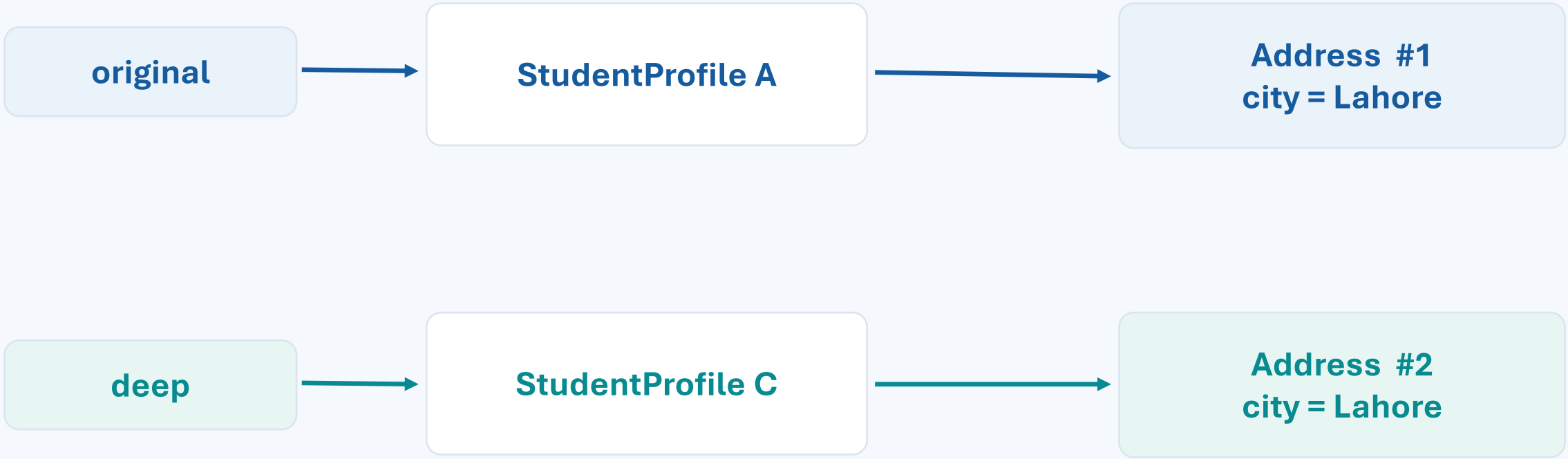
Both references point to object #1.

Shallow copy: new outer object, shared inside



Editing Address #1 affects both students.

Deep copy: independent nested mutable objects



Changing Address #1 does not change Address #2.

Copy constructors communicate the intention

```
class Address {
    private String city;
    Address(String city) { this.city = city; }
    Address(Address a) { this(a.city); }
}
class StudentProfile {
    private String name;
    private Address address;
    StudentProfile(String n, Address a) {
        this.name = n; this.address = a;
    }
    StudentProfile(StudentProfile p) {
        this(p.name, new Address(p.address));
    }
}
```

Predict: which copy stays in Lahore?

```
StudentProfile original = new StudentProfile(
    "Amina", new Address("Lahore"));
StudentProfile alias = original;
StudentProfile shallow = original.shallowCopy();
StudentProfile deep = new StudentProfile(original);

original.getAddress().setCity("Karachi");
```

Predict four city values

original ? alias ? shallow ? deep ?

Arrays have the same reference trap

```
int[][] original = {{1, 2}, {3, 4}};
int[][] b = original.clone();
b[0][0] = 99;
System.out.println(original[0][0]);
```

Why the output is 99

The outer array was copied, but its rows are shared.

For independent rows

Clone each nested row (or copy its elements).

What about Object.clone()?

It exists — but be careful

Object.clone() is protected. The standard mechanism involves Cloneable and a field-by-field (shallow) copy. It does not automatically deep-copy mutable nested objects.

Practical choice for this course

Prefer a clear copy constructor or an explicit copy method. Decide which nested mutable objects must be copied.

Cloneable and exceptional cloning behavior can be studied later.

A practical decision checklist

Need readable logs?	Override toString().
Need domain equality?	Implement equals(Object) and hashCode() together.
Need distinct references?	Use == on objects.
Need a safe new object?	Define a copy constructor; copy nested mutable state.
Need runtime class name?	Call getClass().getSimpleName().

In-class coding task: a better Book class

```
class Book {
    private String isbn;
    private String title;
    // constructor supplied in starter
file
}
```

Required behavior

Two books with the same ISBN compare equal, even if their titles differ. Printing a Book shows both fields.

Stretch: add a Book(Book other) constructor and explain which fields may be shared.

Exit ticket: five decisions, one minute

- If two references point to the very same object, which operator detects it?
- If two distinct objects represent one registration number, which method helps?
- What must accompany an overridden equals()?
- Why can a copied Person still share an Address?
- Which copying technique will you choose for mutable nested fields?

References & code for continued practice

01 Java Object as a Superclass — https://dev.java/learn/inheritance/objects/

02 Oracle Object API — https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Object.html

03 Oracle Secure Coding Guidelines — https://www.oracle.com/java/technologies/javase/seccodeguide.html

04 Runnable examples: ObjectMethodsDemo, EqualityDemo, CopyDemo, ArrayCopyDemo

Takeaway: “Same reference”, “same value”, and “independent copy” are three different claims.